

Experiment No. – 2							
Date of Performance:	29/07/2026						
Date of Submission:	30/07/2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 2 Implementation of Functions

2.1 Aim: Implementation of Functions: A software company is developing a Number Analysis Utility for students to explore different mathematical properties of a number. The application should display a menu and allow the user to select the operation they want to perform.

Write a menu-driven Java program that accepts a number from the user and provides the following options:

1. Calculate the factorial of the number using a method named factorial().
2. Check whether the number is an Armstrong number using a method named testArmstrong().
3. Check whether the number is a palindrome using a method named testPalindrome().
4. Check whether the number is a prime number using a method named testPrime().
5. Display the Fibonacci series up to the specified number of terms using a method named fibonacciSeries().
6. Exit the program.

2.2 Course Outcome: Apply fundamental Java programming concepts, including data types,

operators, control structures, and functions, to develop basic programs.

2.3 Learning Objectives: Use class and its methods in other subclasses, and to show how method overloading is an efficient way to write a program.

2.4 Requirement: Java development kit (JDK as per OS)

2.5 Related Theory: A **function**, known as a **method** in Java, is a named block of code that performs a specific task. Methods help divide a large program into smaller, manageable modules, making the program easier to understand, develop, test, and maintain. Instead of writing the same code multiple times, a method can be created once and called whenever required, thereby promoting code reusability.

Java provides two types of methods:

- **Predefined (Built-in) Methods:** Methods that are already available in Java libraries, such as `Math.sqrt()` and `System.out.println()`.
- **User-defined Methods:** Methods created by the programmer to perform specific tasks according to the application requirements.

A method can accept input values called **parameters** and may return a value after performing its operation. If a method does not return any value, it is declared with the `void` return type. Methods can be invoked (called) from the `main()` method or from other methods whenever their functionality is required.

General Syntax of a Method

```
accessModifier static returnType methodName(parameterList)
{
    // Statements
}
```

Example:

```
public static void display()
{
    System.out.println("Welcome to Java");
}
```

Method Call:

```
display();
```

Methods play an important role in **modular programming**, where a program is divided into independent modules, each responsible for a particular task. This approach improves the structure of the program and simplifies debugging, testing, and maintenance.

Advantages of Methods

- Promotes code reusability.
- Reduces code duplication.

- Improves readability and maintainability.
- Simplifies debugging and testing.
- Supports modular programming.
- Makes large programs easier to develop and manage.
- Enhances program organization by separating different tasks into individual methods.

In this experiment, user-defined methods are used to implement different operations in a menu-driven application. Each operation is performed by a separate method, demonstrating the concepts of method definition, method invocation, and modular programming in Java.

2.6 Procedure:

Start the program by importing the `java.util.Scanner` package to accept user input. Create a Java class and define the required user-defined methods. In the `main()` method, create a `Scanner` object and display a menu containing the available operations. Accept the user's choice and the required input value. Use a `switch` statement to call the appropriate method based on the selected option. Execute the selected method, display the result, and return to the menu. Repeat this process using a `do-while` loop until the user selects the **Exit** option. Finally, close the `Scanner` object and terminate the program.

2.7 Program and Output:

```
import java.util.Scanner;

public class Main {

    // Factorial Method
    public static void factorial(int n) {
        long fact = 1;

        if (n < 0) {
            System.out.println("Factorial is not defined");
            return;
        }
    }
}
```

```

    for (int i = 1; i <= n; i++) {
        fact *= i;
    }

    System.out.println(fact);
}

// Armstrong Method
public static void testArmstrong(int n) {
    int original = n;
    int sum = 0;
    int digits = String.valueOf(n).length();

    while (n > 0) {
        int rem = n % 10;
        sum += (int) Math.pow(rem, digits);
        n /= 10;
    }

    if (sum == original)
        System.out.println("Armstrong Number");
    else
        System.out.println("Not an Armstrong Number");
}

// Palindrome Method
public static void testPalindrome(int n) {
    int original = n;

```

```

int reverse = 0;

while (n > 0) {
    int rem = n % 10;
    reverse = reverse * 10 + rem;
    n /= 10;
}

if (reverse == original)
    System.out.println("Palindrome Number");
else
    System.out.println("Not a Palindrome Number");
}

// Prime Method
public static void testPrime(int n) {

    if (n <= 1) {
        System.out.println("Not a Prime Number");
        return;
    }

    boolean prime = true;

    for (int i = 2; i <= Math.sqrt(n); i++) {
        if (n % i == 0) {
            prime = false;
            break;
        }
    }
}

```

```

    }

    if (prime)
        System.out.println("Prime Number");
    else
        System.out.println("Not a Prime Number");
}

```

```

// Fibonacci Method
public static void fibonacciSeries(int terms) {

    int first = 0, second = 1;

    for (int i = 1; i <= terms; i++) {
        System.out.print(first + " ");
        int next = first + second;
        first = second;
        second = next;
    }
}

```

```

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int choice = sc.nextInt();
    int number = sc.nextInt();

    switch (choice) {

```

```
case 1:
    factorial(number);
    break;

case 2:
    testArmstrong(number);
    break;

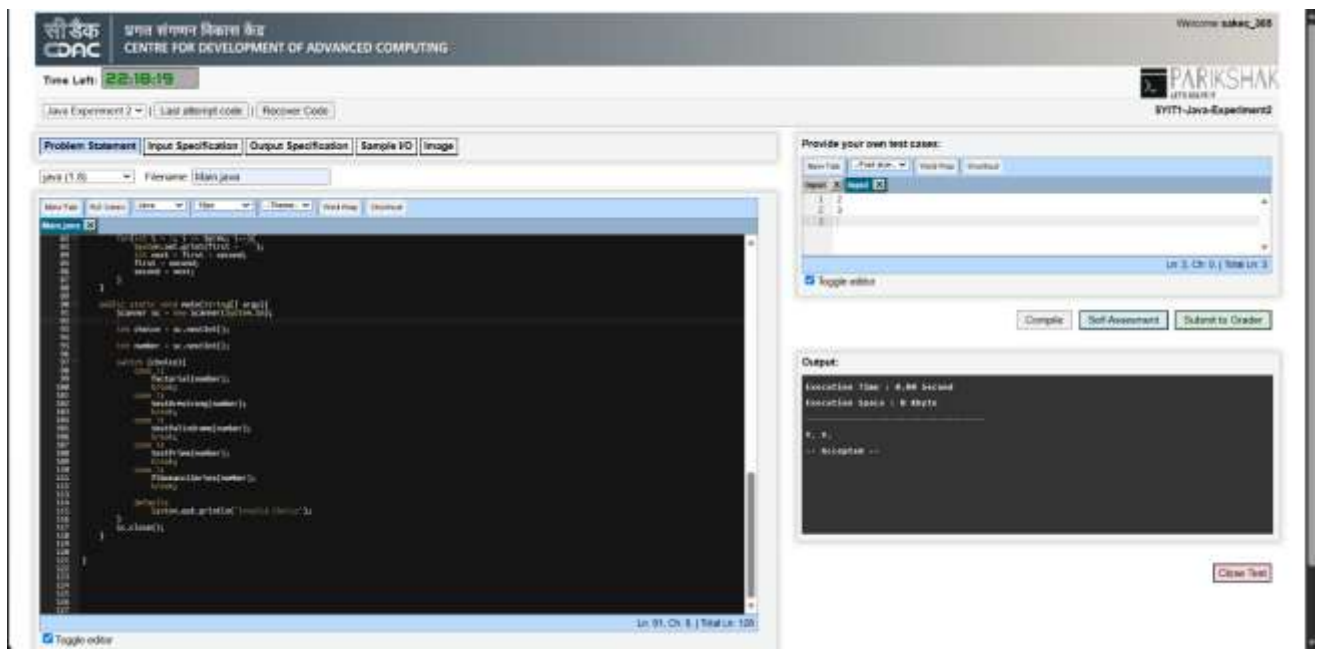
case 3:
    testPalindrome(number);
    break;

case 4:
    testPrime(number);
    break;

case 5:
    fibonacciSeries(number);
    break;

default:
    System.out.println("Invalid Choice");
}

sc.close();
}
}
```



2.8 Conclusion:

This Java program successfully demonstrates the use of methods, loops, conditional statements, and switch-case to perform different mathematical operations. Based on the user's choice, it can calculate the **factorial** of a number, check whether a number is an Armstrong number, palindrome, or prime number, and generate the **Fibonacci series**. The program accepts user input using the `Scanner` class and calls the appropriate method

through a `switch` statement. It is well-organized, easy to understand, and shows the concept of modular programming by dividing each operation into a separate method, making the code more reusable, readable, and efficient.

2.9 Questions:

1. What is Method overloading in java.
2. What is application of method overloading
3. What is polymorphism and it can achieved in java other than method overloading.

Ans 1. Method overloading is a feature in Java where **two or more methods in the same class have the same name but different parameters** (different number, type, or order of parameters). It is an example of **compile-time polymorphism** because the compiler decides which method to call based on the arguments passed.

Ans 2

Method overloading is used to:

- Improve **code readability** by using the same method name for similar operations.
- Increase **code reusability** and reduce the need for different method names.
- Allow methods to work with **different types of data**.
- Make programs **easier to maintain** and understand.
- Provide flexibility by allowing the same operation with different numbers of arguments.

Example: A `sum()` method can add two integers, three integers, or two decimal numbers using overloaded methods.

Ans 3

Polymorphism means "**one name, many forms.**" It allows the same method or object to perform different actions depending on the situation.

There are two types of polymorphism in Java:

1. **Compile-time Polymorphism** – Achieved using **Method Overloading**.
2. **Run-time Polymorphism** – Achieved using **Method Overriding**.

Method Overriding is another way to achieve polymorphism. In method overriding, a subclass provides its own implementation of a method that is already defined in the parent class. The method that gets executed is decided at **runtime**.

Example code:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```